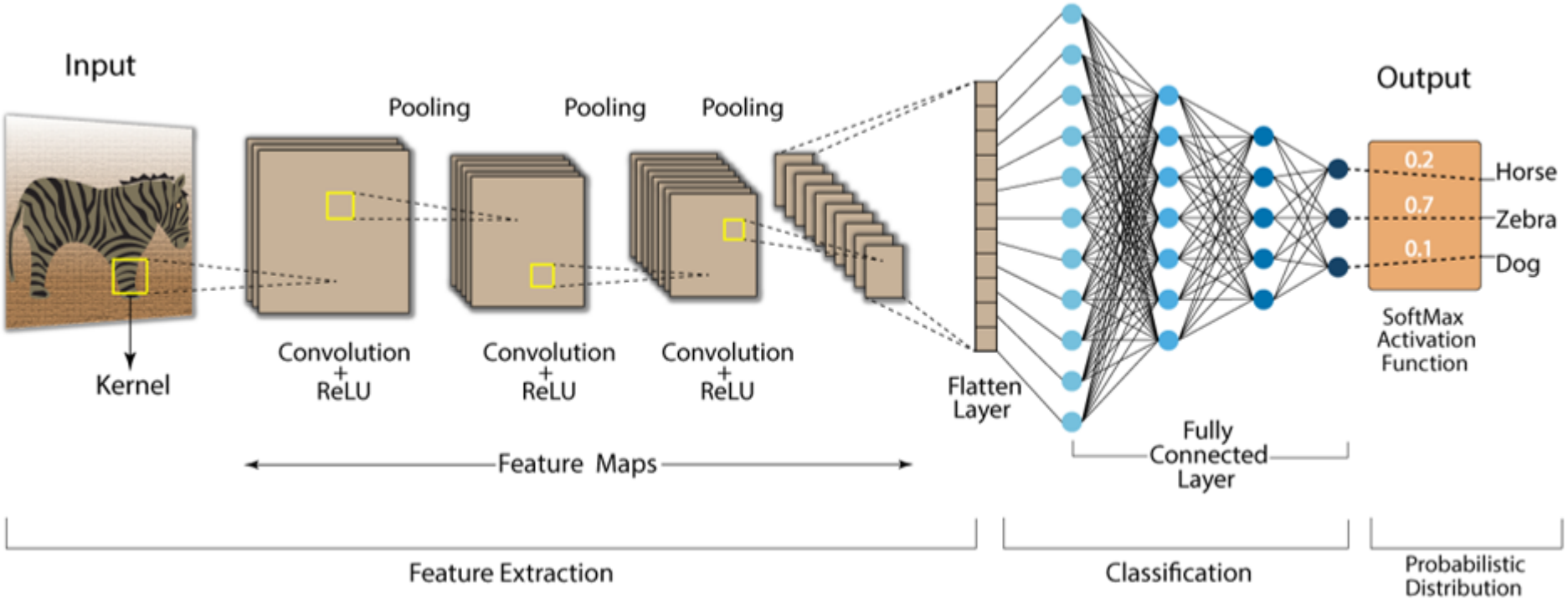
Popular CNN Architectures

CONVOLUTIONAL NEURAL NETWORKS - REVIEW

Convolutional Neural Networks (CNNs) aim to extract features automatically from unstructured data such as images

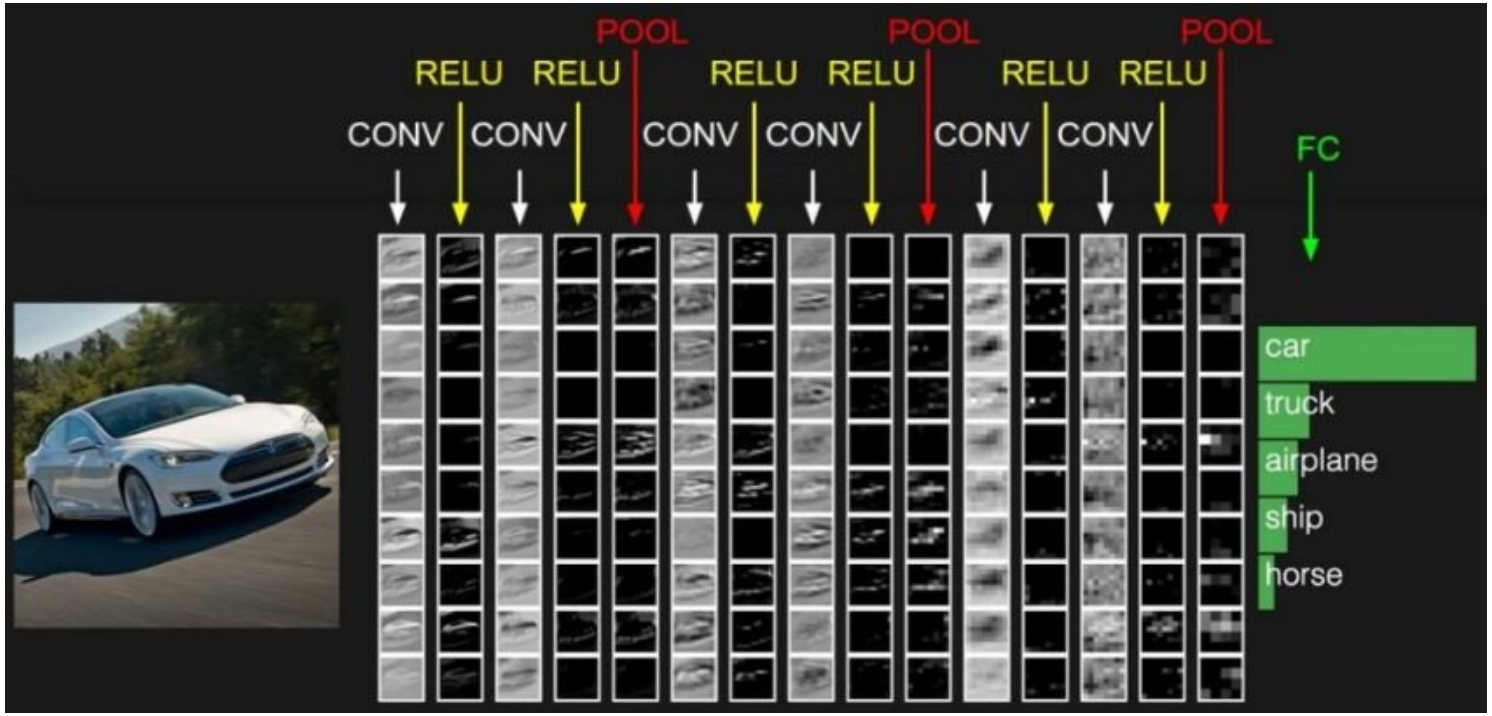
This is particularly useful for visual datasets such as images or videos, where data patterns play a crucial role



CNN ARCHITECTURE

A typical CNN consists of:

- Convolutional layers → feature extraction
- Pooling layers → spatial down sampling
- Fully connected layers → task-specific decision making



POPULAR CNN ARCHITECTURES

There are a lot of CNN architectures present today

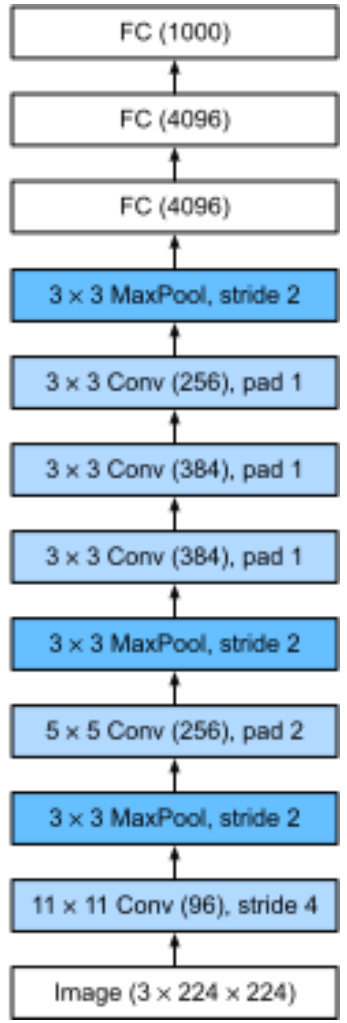
- ImageNet Large Scale Visual Recognition Competition is the acid test for CNN-based architectures
- Some of the popular ones are:
  - AlexNet
  - VGGNet
  - GoogleNet
  - ResNet
- The depth of neural networks has been increasing (from AlexNet with 8 layers to ResNet with 152 layers)
- However, training networks with such a huge amount of layers requires significant resources such as parallelly distributed multi-GPUs
- These networks require to be trained on millions of images and are great at extracting generic features from a vast variety of images

AlexNet

- AlexNet was one of the first architectures to achieve extraordinary results (~17% error rate)
- It had 8 learnable layers: 5 conv + 3 dense
- A distinct feature was that it used kernels of large sizes like (11, 11) and (5, 5) and it was the first to use dropout layers – having 60M parameters
- The entire structure can be written as:

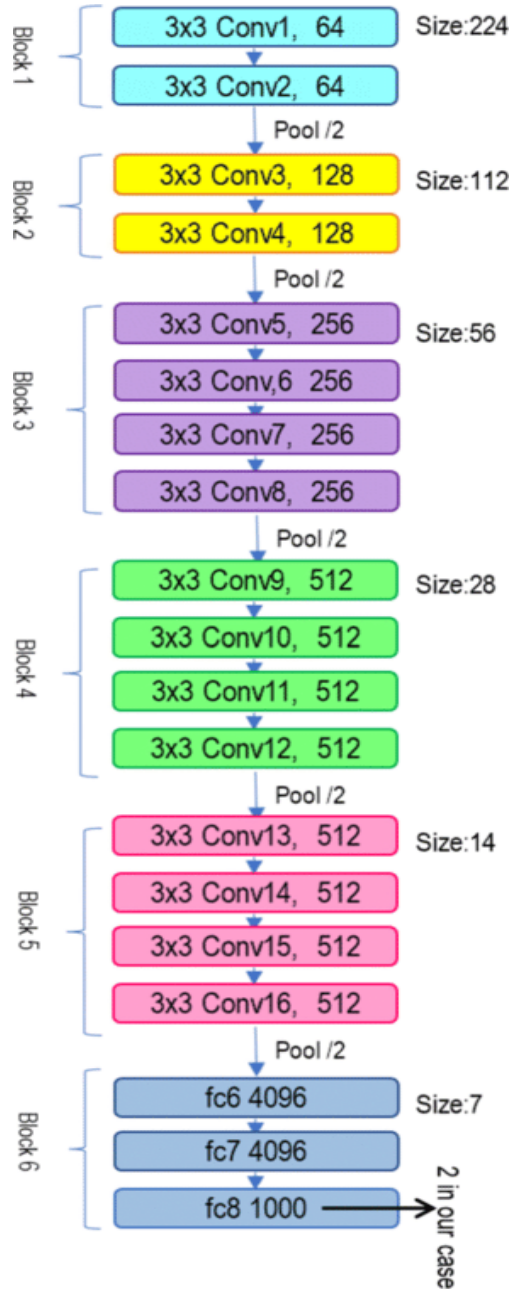
$$(CONV \rightarrow RN \rightarrow MP)^2 \rightarrow (CONV^3 \rightarrow MP) \rightarrow (FC \rightarrow DO)^2 \rightarrow Linear \rightarrow softmax$$

  - *CONV* = convolutional layer (with ReLU activation)
  - *RN* = local response normalisation
  - *MP* = max-pooling
  - *FC* = fully connected layer (with ReLU activation)
  - *Linear* = fully connected layer (without activation)
  - *DO* = dropout



VGGNet

- VGGNet succeeded AlexNet in the ImageNet challenge (~8% error rate)
- Increase in depth from AlexNet – 16 and 19 layer variants (3 dense)
- VGG-16 had 138M and VGG-19 had 144M parameters
- Decreased the kernel size to (3, 3)
- Reduced kernel size but stacked up layers together to not lose out on receptive field → gave richer features than AlexNet



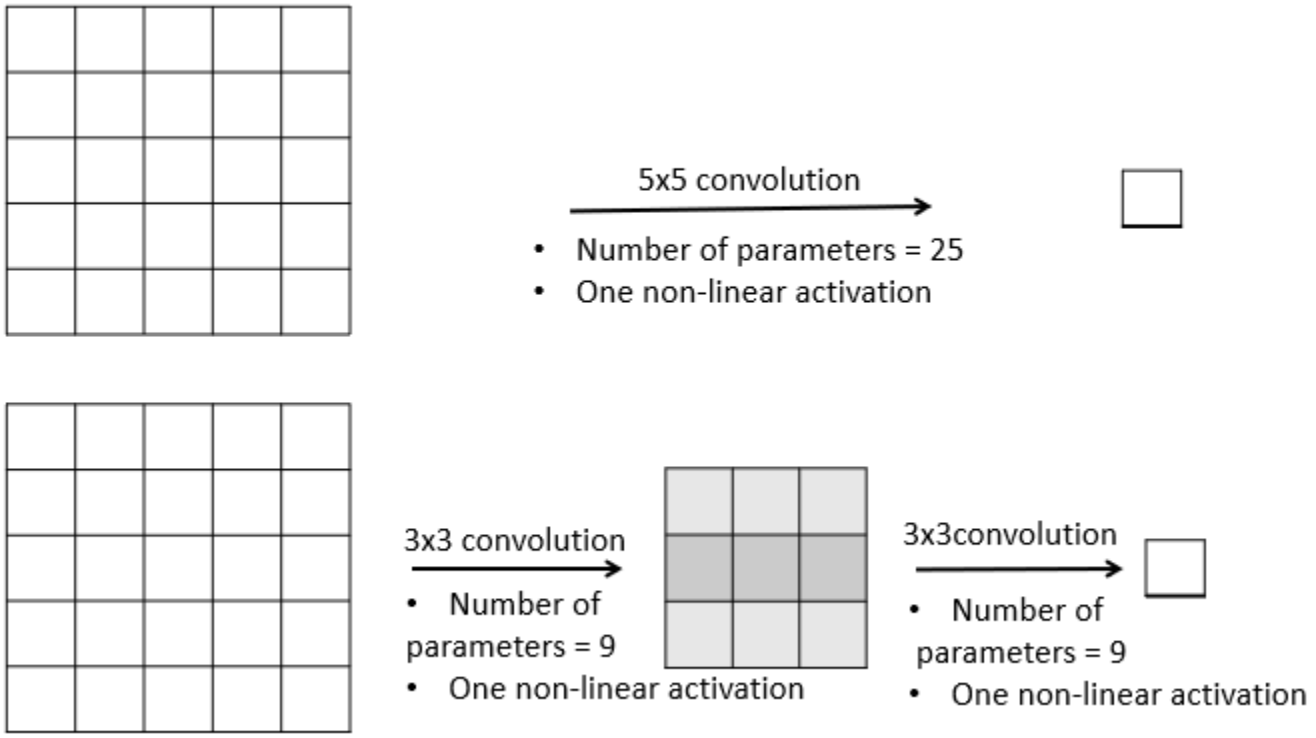
EFFECTIVE RECEPTIVE FIELD

Receptive field is the region of the input (e.g. image pixels) that influences the activation of a neuron in a given layer

The key idea in moving from AlexNet to VGGNet was to increase the depth of the network by using smaller filters

E.g., here we say that the stack of two (3, 3) filters has the same effective receptive field as that of one (5, 5) filter

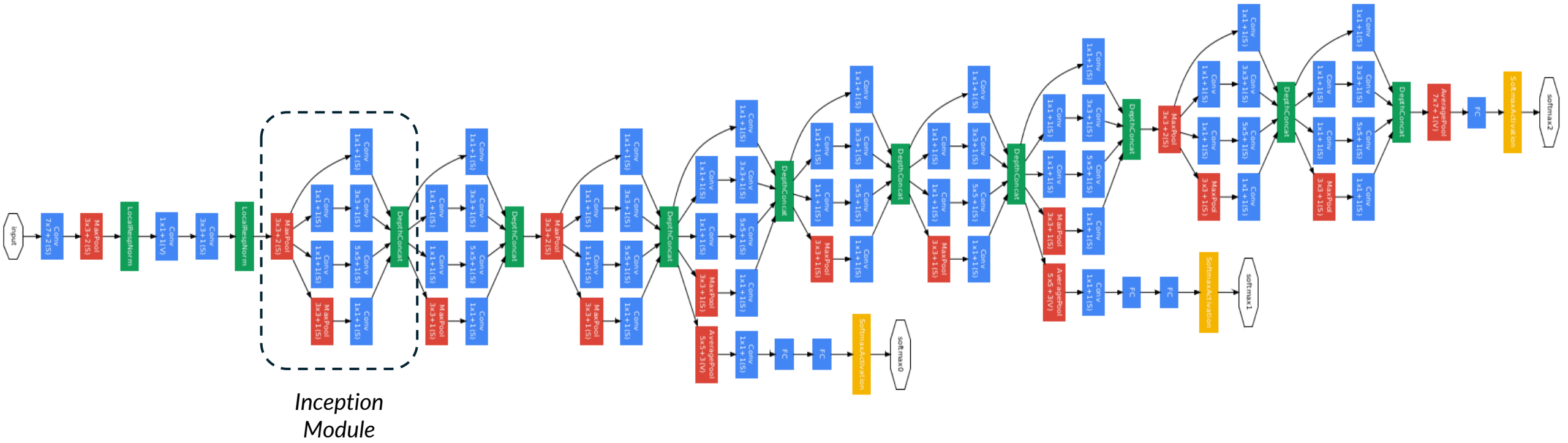
- This is because both these convolutions produce the same output (of size 1 x1 here) whose receptive field is the same 5 x 5 image
- With a smaller (3, 3) filter, we can make a deeper network with more non-linearities and fewer parameters



GoogleNet

- GoogleNet won the ILSVRC'14 challenge with an error rate of about 6.7%
- Unlike the previous innovations, GoogleNet increased the depth using a new type of convolution technique using the **inception module**
- Inception modules process the input in parallel with multiple filter sizes (1×1, 3×3, 5×5) and pooling, then concatenate the results to capture multi-scale features efficiently
- Using a different kernel sizes allowed feature extraction at multiple levels of granularity; these convolutions happened **in parallel**
- This inception module can be called using libraries as a ready-made unit
- GoogleNet stacked these modules, with a total 22 layers, and pooling operations of 3x3 – reducing parameters to 4M
- It did not include FC layers, except for the last SoftMax layer
- Two auxiliary losses in the earlier layers helped counter the vanishing gradients

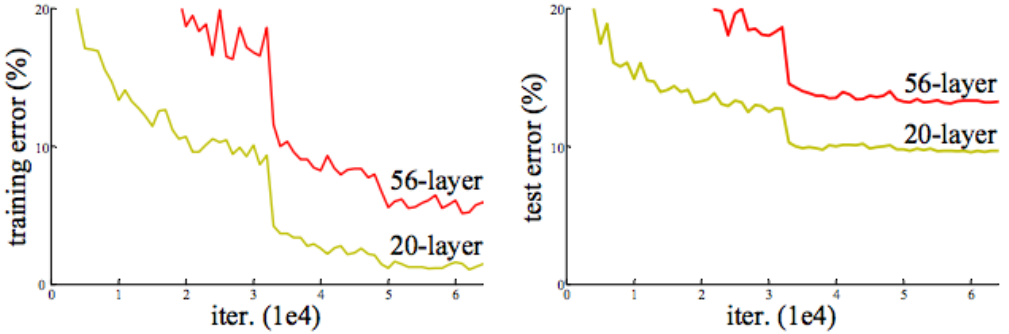
GoogleNet

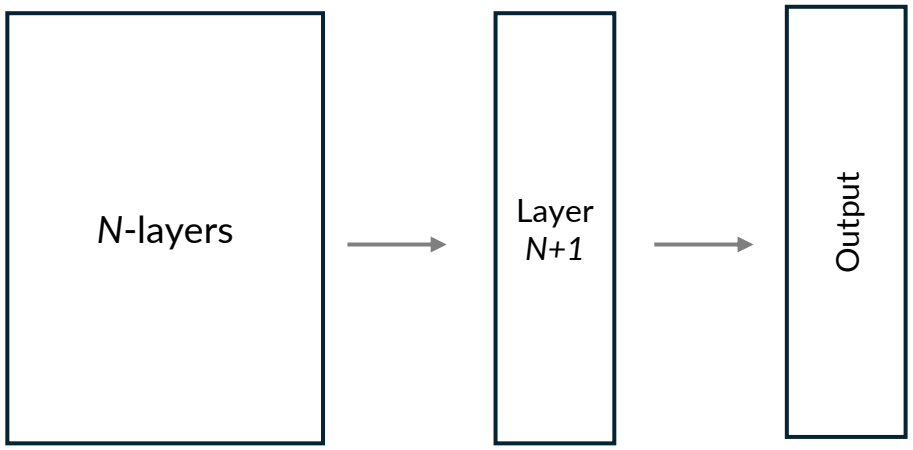


Inception Module

FURTHER INCREASE IN DEPTH

- Until GoogleNet, the most significant improvements in deep learning had appeared in the form of increased network depth
- However, a team of researchers experimented with substantially deeper networks and found contradicting results
- A 56-layered network performed worse than a 20 layered network in the CIFAR-10 dataset (a benchmark) – this was not due to overfitting
- Another issue with increasing depth was exploding or vanishing gradients
- Ensuring that addition of layers does not give worse results was the motivation behind **ResNet**

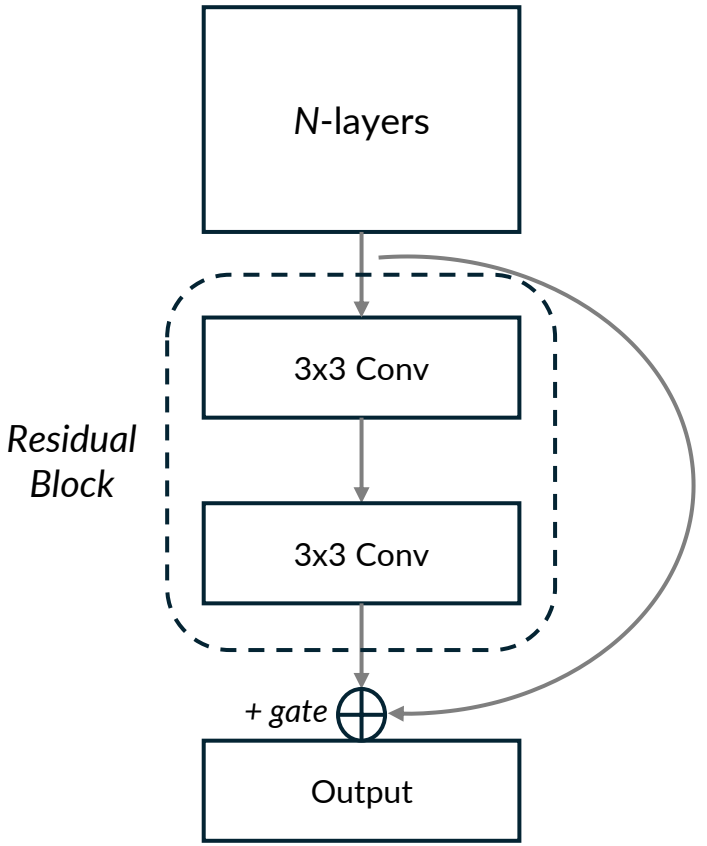




Conceptually, an additional layer between the convolutional layers and final layer should extract more features and give better performance

ResidualNet

- The ResNet team produced an architecture with **skip connections** which enabled training of networks as deep as 150 layers
- This gave groundbreaking results with a 3.57% error rate on ImageNet
- With skip connections, we check the performance of additional layers during training (with gradients), and make the weights of the layers in the residual block 0, if the performance is not better than the previous N-layers
- This is checked using a *+ gate* which compares the performance of *N+2* and *N* layers
- ResNet Architecture:
  - 35, 50, 101, 152-layer variants
  - Residual blocks: 2 conv layers of 3x3
  - Only one 1000 FC SoftMax layer for classification
  - Global average pooling after last conv layer
  - Batch normalisation after every conv layer
  - No dropout
  - Stochastic GD + Momentum for learning



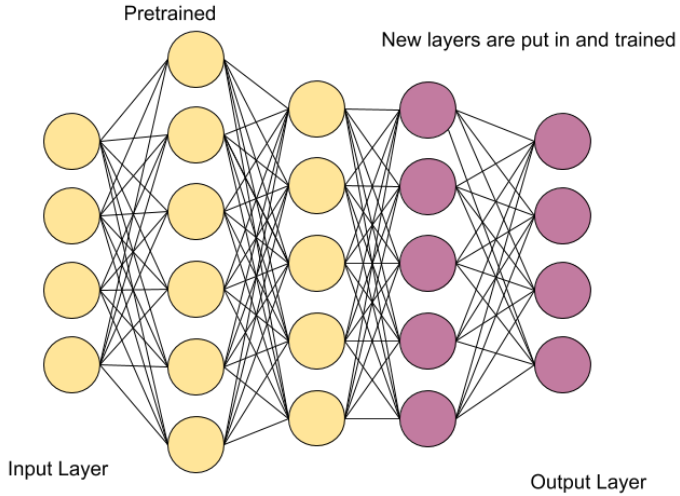
Transfer Learning

TRANSFERRING LEARNT KNOWLEGDE

- Training a deep neural network needs an immense amount of resources
- We have discussed multiple CNN based networks which were trained on millions of images of various classes
- The ImageNet dataset itself has about 1.2 million images of 1000 classes
- CNNs are basically feature-extractors
  - The convolutional layers learn a representation of an image, which can then be used for any task such as classification, object detection etc.
  - Generally, early CNN layers learn generic features (edges, textures etc.) while deeper layers learn task-specific abstractions
- Is there a way we can “transfer” this knowledge to solve other problems as well, where heavy resources and large data are not available?

TRANSFER LEARNING

- Many vision tasks share low-level and mid-level visual patterns
- Pretrained models enable capture of these general visual representations (*feature vectors*) in early layers
- Transfer learning reuses learned representations instead of relearning from scratch
- Knowledge learned from a source task is adapted to a target task
- Transfer learning mainly reuses the feature extraction backbone
- It enables **faster convergence** and **better performance with limited data**



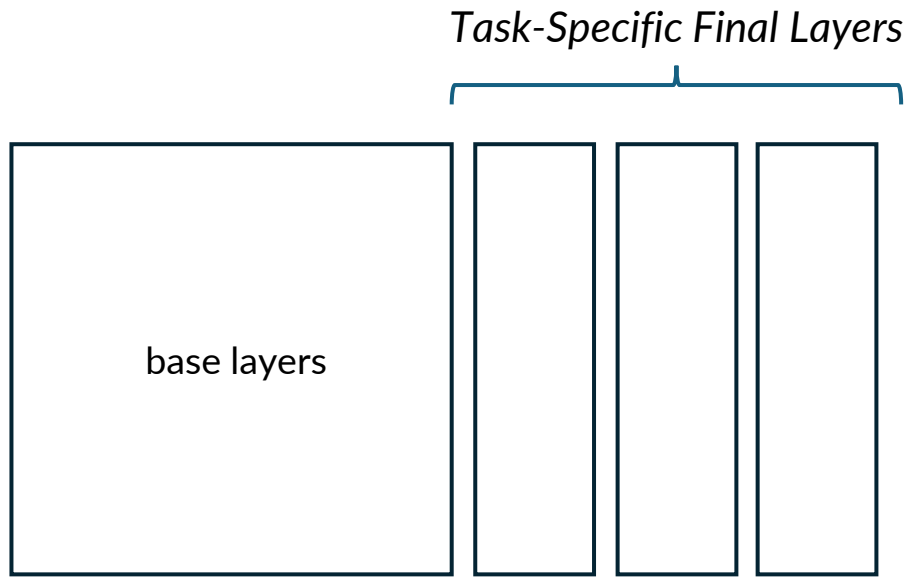
We can either use the layer structure or the learned weights of the source model (or both)

Two common strategies for transfer learning are:

- **Full-retraining** (fine-tuning of all layers) of a learned model
- **Partial-retraining** (by freezing some layers) of a learned model

FEATURE VECTORS

- CNNs transform images into high-dimensional feature vectors (numerical representation of content)
- CNNs are often used as feature extractors:
  1. Pass image through pretrained CNN
  2. Remove classification head
  3. Extract feature vectors
  4. Train a separate ML classifier
- Feature Vectors encode semantic information (shape, texture, object parts) and reduce raw pixel dimensionality



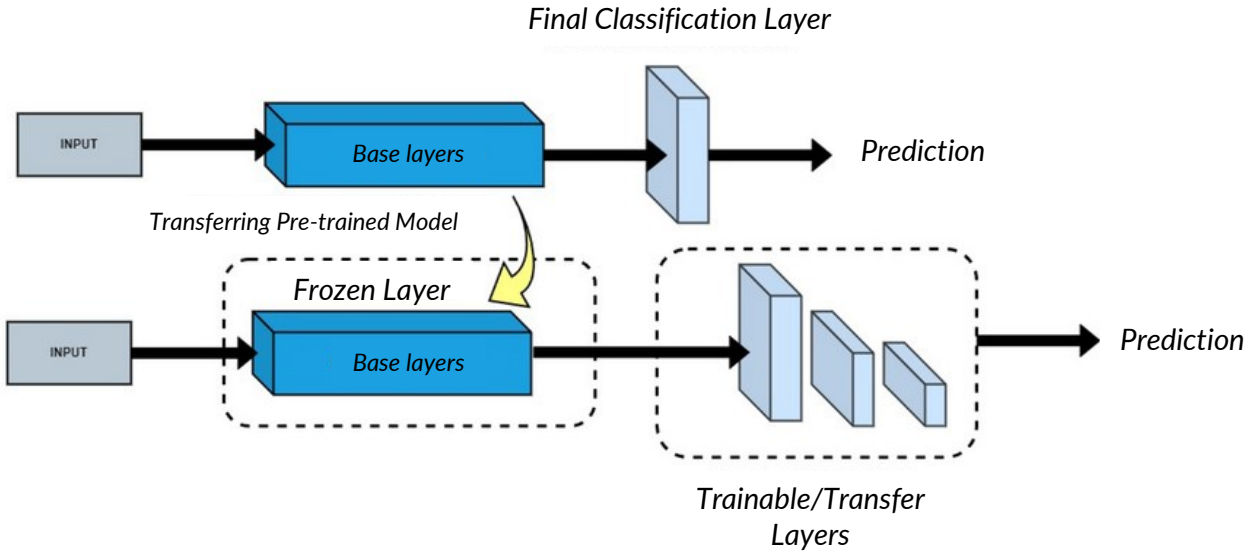
For example, if the last few layers of the network are task specific, we can discard them and use the generic layers for gaining an abstract understanding

USE CASES OF TRANSFER LEARNING

- Transfer learning is not limited to classification tasks
- We may want to use the feature extraction capabilities of a model for a variety of tasks:
  - Calculating the forest cover in a satellite image
  - Medical imaging (X-Rays, MRIs)
  - Industrial defect detection
  - Facial recognition
- We reuse the network's top-level feature extraction to obtain generic representations for downstream tasks
- This is not specific to images, e.g.,
  - Learned layers of a NN transform words to vector representations (embeddings) for an English NLP task
  - These embeddings can be used and fine-tuned for Hindi NLP related tasks as well

PARTIAL TRANSFER LEARNING

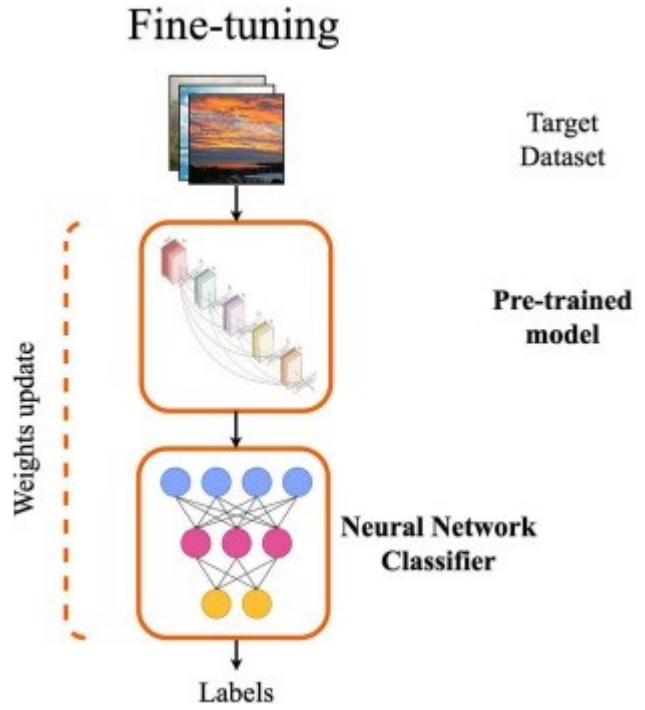
- '**Freeze**' the initial layers, i.e. use the same weights and biases that the network has learnt
- Remove the last few layers of the pre-trained model, add new layer(s) at the end and train only the newly added layer(s):
  - Freeze early layers, fine-tune deeper ones
  - Gradually unfreeze layers to optimise
  - Lower learning rates for pretrained layers
- This assumes pretrained features are already relevant
- We use partial retraining when the **available data is less** and the **target task is similar to source task**



Reusing layers gives faster training and lower risk of overfitting

FULL RETRAINING (FINE-TUNING)

- Retrain all the weights starting (initialising) from the weights and biases that the net has already learnt
  - We don't want to unlearn a large fraction of what the pre-trained layers have learnt
  - So, for the initial layers, we choose a low learning rate
- We use fine-tuning when the **available data is large** and the **target task is significantly different from the source task**
- To choose transfer strategy, consider:
  - Dataset size
  - Domain similarity with source
  - Compute constraints
  - Model deployment requirements



Trade-off here is better adaptation vs higher overfitting risk